Classes In Java

* Type Of Classes

- Concrete Class
- Abstract Class
- Super Class and Sub Class
- Object Class
- Nested Class
  - Inner Class (Non Static Nested Class)
  - Anonymous Inner Class
  - Member Inner Class
  - Local Inner Class
  - Static Nested Class / Static Class
- Generic Class
- POJO Class
- Enum Class
- Final Class
- Singleton Class
- Immutable Class
- Wrapper Class

⇒ Concrete Class:

- These are those class that we can create an instance using NEW keyword.
- All the methods in this class have implementation.
- It can also be your child class from interface or extend abstract class.
- A class access modifier can be "public" or "package private" (no explicit modifier defined)

For example :-

```java
public class Person {

    int empID;

    Person(int empID){
        this.empID = empID;
    }

    public int getEmpID(){
        return empID;
    }
}
```

```java
public interface Shape {
    public void computeArea();
}
```

```java
public class Rectangle implements Shape{

    @Override
    public void computeArea() {
        System.out.println("Compute Rectangle Are
    }
}
```

⇒

Abstract Class (0 to 100% Abstraction):

Show only important features to users and hide its internal implementation.
2 ways to achieve abstraction:

- Class is declared as abstracted through keyword "abstract".
- It can have both abstract(method without body) and non abstract methods.
- We can not create an instance of this class.
- We parent has some features which all child classes have in common, then this can be used.
- Constructors can be created inside them. And with super keyword from child classes we can access them.

For Example :

```java
public abstract class Car {

    int mileage;

    Car(int mileage){
        this.mileage = mileage;
    }

    public abstract void pressBreak();

    public abstract void pressClutch();

    public int getNumberOfWheels(){
        return 4;
    }
}
```

→ Abstract Method

→ Abstract Method

→ Non-Abstract Method

```java
public abstract class LuxuryCar extends Car{

    LuxuryCar(int mileage){
        super(mileage);
    }

    public abstract void pressDualBreakSystem();

    @Override
    public void pressBreak(){
        //implementation of it goes here
    }
```

* Another abstract class inheriting above abstract class

→ Additional abstract method

```java
public class Audi extends LuxuryCar{

    Audi(int mileage){
        super(mileage);
    }

    @Override
    public void pressClutch() {
        //its implementation goes here
    }

    @Override
    public void pressDualBreakSystem() {
        //its implementation goes here
    }
}
```

* Concrete class inheriting abstract class (LuxuryCar)

Both abstract methods are implemented

⇒

Super and Sub Class:

- A class that is derived from another class is called a Subclass.
- And from class through which Subclass is derived its called Superclass.
- In Java, in the absence of any other explicit superclass, every class is implicitly a subclass of **Object** class.
- Object is the topmost class in Java.

It has some common methods like clone(), toString(), equals(), notify(), wait() etc…

For Example:

```java
public class ObjectTest {

    public static void main(String[] args){

        ObjectTest obj = new ObjectTest();

        Object obj1 = new Person( emplD: 1);
        Object obj2 = new Audi( mileage: 10);

        System.out.println(obj1.getClass());
        System.out.println(obj2.getClass());
    }
}
```

```
class Person
class Audi

Process finished with exit code 0
```

* Since Object is the parent class of every class in Java & we know that reference of child class can be kept in parent class, hence Audi & Person objects can be kept in reference of Object.

⇒ **Nested Class:**

Class within another class is called Nested Class.

When to use?
If you know that, a class(A) will be used by only one another class(B), then instead of created new file(A.java) for it, we can create nested class inside B class itself.
And
Also help us to group logically related classes in one file.

Scope:
Its Scope is same as of its Outer class.

It is of 2 types:
- Static Nested Class
- Non Static Nested Class
  - Member Inner Class
  - Local Inner Class
  - Anonymous Inner Class

Now let's go through types of nested classes one by one:

⇒ **Static Nested Class:**

- It do not have access to the non static instance variable and method of outer class.
- Its object can be initiated without initiating the object of outer class.
- It can be private, public, protected or package-private(default, no explicit declaration)

For Example:

```
class OuterClass {

    int instanceVariable = 10;
    static int classVariable = 20;

    static class NestedClass{

        public void print(){
            System.out.println(classVariable + instanceVariable);
        }
    }
}
```

Error because Static inner class can only access static variable / methods

```java
public class ObjectTest {

    public static void main(String args[]) {

        OuterClass.NestedClass nestedObj = new OuterClass.NestedClass();
        nestedObj.print();
    }
}
```

Since static classes can be accessed directly with the name of the class. So we'll need the name of outerclass. Hence object of outer class was not needed.

* Nested Classes can be created with any type of access modifiers i.e. public, private, protected, default, package-protected

Let's take the example of a nested class with private access modifier

* Nested class object can be created within the same class itself.

```java
class OuterClass {

    int instanceVariable = 10;
    static int classVariable = 20;

    private static class NestedClass{

        public void print(){
            System.out.println(classVariable);
        }
    }

    public void display(){
        NestedClass nestedObj = new NestedClass();
        nestedObj.print();
    }
```

```java
public class ObjectTest {

    public static void main(String args[]) {

        OuterClass outerClassObj = new OuterClass();
        outerClassObj.display();
    }
}
```

So, to access the private nested class, we had to create an object within the class itself & then expose

⇒

Inner Class or Non Static Nested Class:

- It have access to all the instance variable and method of outer class.
- Its object can be initiated on after initiating the object of outer class.

1. Member Inner Class:
   - its can be private, public, protected, default

For Example :

```
class OuterClass {

    int instanceVariable = 10;
    static int classVariable = 20;

     class InnerClass{

        public void print(){
            System.out.println(classVariable + instanceVariable);
        }
    }
}
```

* To invoke it, we'll need an object of outer class

```
public class ObjectTest {

    public static void main(String args[]) {

        OuterClass outerClassObj = new OuterClass();

        OuterClass.InnerClass innerClassObj = outerClassObj.new InnerClass();
        innerClassObj.print();

    }
```

⇒

2. Local Inner Class:

- These are those classes which are defined in any block like for loop, while loop block, If condition block, method etc.
- It can not be declared as private, protected, public. Only default(not defined explicit) access modifier is used.
- It can not be initiated outside of this block.

For Example:

```java
class OuterClass {

    int instanceVariable = 1;
    static int classVariable = 2;


    public void display(){

        int methodLocalVariable = 3;

        class LocalInnerClass{
            int localInnerVariable = 4;

            public void print(){
                System.out.println(instanceVariable + classVariable + methodLocalVariable + localInnerVariable);
            }
        }

        LocalInnerClass localObj = new LocalInnerClass();
        localObj.print();

    }
}
```

So, it can be invoked inside the block only. As soon as the scope of block ends, it's scope also ends.

* Inheritence in Nested Classes
- Example - 1: One inner class can inherit another inner class in same outer class.

```java
class OuterClass {

    int instanceVariable = 1;
    static int classVariable = 2;


    class InnerClass1{

        int innerClass1 = 3;
    }

    class InnerClass2 extends InnerClass1{

        int innerClass2 = 4;

        void display(){
            System.out.println(innerClass1 + innerClass2 + instanceVariable + classVariable);
        }
    }

}
```

```java
public class ObjectTest {

    public static void main(String args[]) {

        OuterClass outerClassObj = new OuterClass();
        OuterClass.InnerClass2 innerClass2Obj = outerClassObj.new InnerClass2();
        innerClass2Obj.display();

    }
}
```

Example - 2: Static inner class inherited by different class

```java
class OuterClass {
                    Nested Class
    static class InnerClass {
        public void display(){        Static Nested Class
            System.out.println("inside InnerClass");
        }
    }
}
```

Nested Class

```java
public class SomeOtherClass extends OuterClass.InnerClass {

    public void display1(){
        display();
    }
}
```

Example - 3: Non Static Inner Class inherited by different class

```java
class OuterClass {

    class InnerClass {
        public void display(){
            System.out.println("inside InnerClass");
        }
    }
}
```

```java
public class SomeOtherClass extends OuterClass.InnerClass {

    SomeOtherClass(){
        new OuterClass().super();
        //as you know, when child class constructor invoked, it first invoked the constructor of parent.
        //but here the parent is Inner class, so it can only be accessed by the object of outerclass only.
    }

    public void display1(){
        display();
    }
}
```

=> Anonymous Inner Class

Anonymous Inner Class:

An inner class without a name called Anonymous class.

Why its used:
- When we want to override the behaviour of the method without even creating any subclass.

For Example:

```java
public abstract class Car {

    public abstract void pressBreak();
}
```

```java
public class Test {

    public static void main(String args[]) {

        Car audiCarObj = new Car() {

            @Override
            public void pressBreak() {
               //my audi specific implementation here
                System.out.println("Audi specific break changes");
            }
        };

        audiCarObj.pressBreak();
    }
}
```

Now we know that we cannot create an object of abstract class but here we had done. Let's understand what happened:

So 2 things happened behind the scene:

- Sub class is created, name decided by the compiler
- Creates an object of sub class and assigns its reference to object.

Similarly for interface also, it works in the same way:

va × Test$1.class × Test.class ×

iled .class file, bytecode version: 52.0 (Java 8)

```
//
// Source code recreated from a .class file by IntelliJ IDEA
// (powered by FernFlower decompiler)
//

final class Test$1 extends Car {
    Test$1() {
    }

    public void pressBreak() {
        System.out.println("Audi specific break changes");
    }
}
```

Generic Classes

* Generic class helps us to write a class in a generic manner that helps to avoid the typecasting that we'll have to use with Object class

Let's see via an example:

```
public class Print {

    Object value;

    public Object getPrintValue(){
        return value;
    }

    public void setPrintValue(Object value){
        this.value = value;
    }
}
```

Since Object is parent of every class, we used it here. Now here value can be of any type i.e String, integer etc. The only issue is that we'll have to typecast it as per our use.

```
public class Main {

    public static void main(String args[]) {

        Print printObj1 = new Print();
        printObj1.setPrintValue(1);
        Object printValue = printObj1.getPrintValue();
        //we can not use printValue directly, we have to typecast it, else it will be compile time er
        if((int)printValue == 1){
            //do something
        }
    }
}
```

So we had to typecast printValue to int to compare it with 1.

* How to define Generic Classes ?

-> We can do it using <T>, this T can be any alphabet like A, B, C etc.

So let's make our previous Print Class generic

```
public class Print<T> {

    T value;

    public T getPrintValue(){
        return value;
    }

    public void setPrintValue(T value){
        this.value = value;
    }
}
```

Generic Type (In above example <T>) can be any non-primitive object

```
public class Main {

    public static void main(String args[]) {

        Print<Integer> printObj1 = new Print<Integer>();
        printObj1.setPrintValue(1);
        Integer printValue = printObj1.getPrintValue();
        if(printValue == 1){
            //do something
        }
    }
}
```

Therefore, while creating printObject1 we replaced T by Integer making it of an Integer type.

⇒ Inheritence In Generic Classes

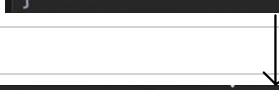
1) Non - Generic Subclass

```java
public class Print<T> {

    T value;

    public T getPrintValue(){
        return value;
    }

    public void setPrintValue(T value){
        this.value = value;
    }
}
```



```java
public class ColorPrint extends Print<String>{

}
```

So, here above we have extended / inherited a Generic Class to a non-generic subclass.

While extending / inheriting a generic class to a non generic subclass, we have to define the type of T at the time of extending itself.

```java
public class Main {

    public static void main(String args[]) {

        ColorPrint colorPrintObj = new ColorPrint();
        colorPrintObj.setPrintValue("2");
    }
}
```

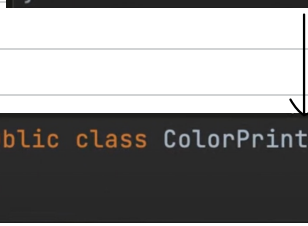
2.) Generic Subclass

```java
public class Print<T> {

    T value;

    public T getPrintValue(){
        return value;
    }

    public void setPrintValue(T value){
        this.value = value;
    }
}
```



```java
public class ColorPrint<T> extends Print<T>{

}
```

```java
public class Main {

    public static void main(String args[]) {

        ColorPrint<String> colorPrintObj = new ColorPrint<>();
        colorPrintObj.setPrintValue("2");
    }
}
```

So, for a non-generic subclass, we can specify the type of T at the time of object creation

* More than one Generics Type Example:

- We can create as many number of generic parameters as we want i.e 1, 2, 3, 4 -- N (Classname $< T_1, T_2, T_3, T_4 ---- T_N >$

For Example:

```java
public class Pair<K,V> {

    private K key;
    private V value;

    public void put(K key, V value){
        this.key = key;
        this.value = value;
    }
}
```

```java
public class Main {

    public static void main(String args[]) {

        Pair<String, Integer> pairObj = new Pair<>();
        pairObj.put("hello", 1243);

    }
}
```

So, here we've used two generic parametes.

* Also both the syntaxes are right:

Pair < String, Integer > object = new Pair < String, Integer > ();

&

Pair < String, Integer > object = new Pair <> ();

✳ **Generic Method:**

What if we only wants to make Method Generic, not the complete Class, we can write Generic Methods too.

- Type Parameter should be before the return type of the method declaration.
- Type Parameter scope is limited to method only.

To make a method generic, put the generic type that we want it to accept before the return Type. For Example:-

```
public class GenericMethod {

    public <K,V> void printValue(Pair<K,V> pair1, Pair<K,V> pair2 ){

        if(pair2.getKey().equals(pair2.getKey())){
            //do something
        }
    }
}
```

* Generic Method with single generic parameter

```
public class Print{


    public <T> void setValue(T busObject){

        //do something
    }

}
```

```
public class Main {

    public static void main(String args[]) {

        Print printObj = new Print();
        printObj.setValue(new Bus());

    }
}
```

* Raw Type Object

It's a name of the generic class or interface without any type argument.

For Example :

```java
public class Print<T> {

    T value;

    public T getPrintValue(){
        return value;
    }

    public void setPrintValue(T value){
        this.value=value;
    }
}
```

```java
public class Main {

    public static void main(String args[]) {

        Print<String> parametrizedTypePrintObject = new Print<>();

        //internally it passes Object as parametrized type.
        Print rawTypePrintObject = new Print();
         rawTypePrintObject.setPrintValue(1);
         rawTypePrintObject.setPrintValue("hello");
    }
}
```

So while creating the object, we didn't pass any type as parameterized type. Therefore, internally it passes Object as parameterized type.

So rawTypePrintObject is a raw type object.

* Bounded Generics

Bounded Generics:
It can be used at generic class and method

- Upper Bound (<T extends Number>) means T can be of type Number or its Subclass only. Here superclass (in this example Number) we can have interface too.
- Multi Bound .

It is used to restrict what all objects we can pass at this type parameter.

For Example:

- Upper Bound

Can also be interface

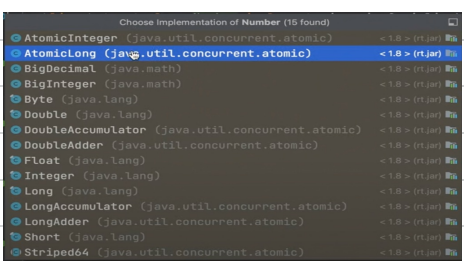
```
public class Print<T extends Number> {

    T value;

    public T getPrintValue(){
        return value;
    }

    public void setPrintValue(T value){
        this.value=value;
    }
}
```

So now only Number or its child/subclasses can be used as objects which in our case are:




```
public class Main {

    public static void main(String args[]) {

        Print<Integer> parametrizedTypePrintObject = new Print<Integer>();
    }
}
```

⇒ Correct since Integer is a child class of number

```
public class Main {

    public static void main(String args[]) {

        Print<String> parametrizedTypePrintObject = new Print<>();
    }
}
```

⇒ Incorrect as String is not a child class of Number.

* **Multi Bound:**
<T extends Superclass & interface1 & interface N>

- **The first restrictive type should be concrete class**
- **2,3 and so on... can be interfaces.**

For Example:

```
public class A extends ParentClass implements Interface1, Interface2{
}
```

So let's say I want the exact above as my type parameter. It'll be:

```
public class Print<T extends ParentClass & Interface1 & Interface2> {

    T value;

    public T getPrintValue() { return value; }

    public void setPrintValue(T value) { this.value = value; }
}
```

So, the below code will work fine

```
public class Main {

    public static void main(String args[]) {

        A obj = new A();
        Print<A> printObj = new Print<>();

    }
}
```

* **Wildcards:**

- **Upper bounded wildcard : <? extends UpperBoundClassName> i.e. class Name and below**
- **Lower bounded wildcard : <? Super LowerBoundClassName> i.e. class Name and above**
- **Unbounded wildcard <?> only you can read**

For Example:

Let's say I have a Vehicle class & Bus, Car are extending it

```
public class Main {

    public static void main(String args[]) {

        List<Vehicle> vehicleList = new ArrayList<>();
        vehicleList.add(new Bus());
        vehicleList.add(new Car());

        List<Bus> busList = new ArrayList<>();

        vehicleList = busList; //NO

        busList = vehicleList; //no


        Vehicle vehicleOb = new Vehicle();
        Bus busObj = new Bus();

        vehicleOb = busObj; // ok

    }
}
```

Here both above assignments are invalid because lists are different from objects

- busList = vehicleList is invalid because in busList we can store only the objects of type Bus.

- vehicleList = busList is invalid because in vehicleList we can store both bus & car objects while busList can contain only bus objects. Hence, it is invalid.

So, here wildcards can help us

```
public class Print{

    public void setPrintValues(List<? extends Vehicle> vehicleList){

    }
}
```

So we've used upperbound wild card. Therefore we can pass Vehicle & its child class

```java
public class Main {

    public static void main(String args[]) {

        List<Vehicle> vehicleList = new ArrayList<>();
        vehicleList.add(new Bus());
        vehicleList.add(new Car());

        List<Bus> busList = new ArrayList<>();


        Print printObj = new Print();
        printObj.setPrintValues(busList);

    }
}
```

```java
public class Main {

    public static void main(String args[]) {

        List<Vehicle> vehicleList = new ArrayList<>();
        vehicleList.add(new Bus());
        vehicleList.add(new Car());

        List<Bus> busList = new ArrayList<>();


        Print printObj = new Print();
        printObj.setPrintValues(vehicleList);

    }
}
```

So now we can pass vehicleList & busList since vehicleList & its child classes are allowed.

* Now let's see lower bound wildcards

```java
public class Print{

    public void setPrintValues(List<? super Vehicle> vehicleList){

    }
}
```

Now we can pass vehicle & its superclass type. Since vehicle is inheriting anything so Object can be used along with vehicle.

```java
List<Object> objList = new ArrayList<>();
printObj.setPrintValues(objList);
```

Now a question arises that can't we use generic type parameter in place of wildcards to achieve the functionality.

Therefor let's see the difference b/w wildcard & generic type

Let's create one wildcard method & one generic type method

```java
public class Print {


    //wild card method
    public void computeList(List<? extends Number> source, List<? extends Number> destination){



    }


    //generic type method
    public <T extends Number> void computeList1(List<T> source, List<T> destination){

    }

}
```

The main difference is that wildcards are a bit less restrictive as compared to generic type. For example, in the above, we can give 2 different type of list as source & destination for the wildcard but for generic method we have to give same type of list to source & destination.

In wildcard we have boundation as well i.e we can use super but in generic method we can't do so.

```java
public class Main {

    public static void main(String args[]) {


        List<Integer> wildCardIntegerSourceList = new ArrayList<>();
        List<Float> wildCardIntegerDesitanationList = new ArrayList<>();

        Print printObj = new Print();
        printObj.computeList(wildCardIntegerSourceList, wildCardIntegerDesitanationList);

        printObj.computeList1(wildCardIntegerSourceList, wildCardIntegerDesitanationList);


    }
}
```

So since compute1 is a generic method & it'll need same types as parameters Therefore showing compilation error in case of different types.

* Unbounded Wildcard
  - Mostly used when my method can work on the methods provided by the object class

* Type Erasure:
  Whatever the code we write, when the bytecode is generated, it is all replaced by actual values.

- Generic Class Erasure:

```java
public class Print <T> {

    T value;

    public  void setValue(T val) {
        this.value = val;
    }
}
```

→

```java
public class Print {

    Object value;

    public  void setValue(Object val) {
        this.value = val;
    }
}
```

- Generic Class Bound Type Erasure:

```java
public class Print<T extends Number> {

    T value;

    public  void setValue(T val) {
        this.value = val;
    }
}
```

→

```java
public class Print{

    Number value;

    public  void setValue(Number val) {
        this.value = val;
    }
}
```

- Generic Method Erasure:

```java
public class Print{

    public <T> void setValue(T val) {
        System.out.println("do something");
    }
}
```

```java
public class Print{

    public void setValue(Object val) {
        System.out.println("do something");
    }
}
```

- Generic Bound Type Method Erasure:

```java
public class Print{

    public <T extends Bus> void setValue(T val) {
        System.out.println("do something");
    }
}
```

```java
public class Print{

    public void setValue(Bus val) {
        System.out.println("do something");
    }
}
```